

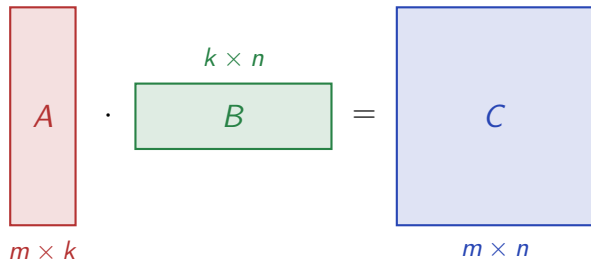
# Cache-Optimal Tiling for Mixed-Precision Matrix Multiplication

Areg Mkhitarian    George Ginzburg

Technion – Israel Institute of Technology

Supervisor: Alon Amid

# Matrix Multiply is Memory-Bound



for example:

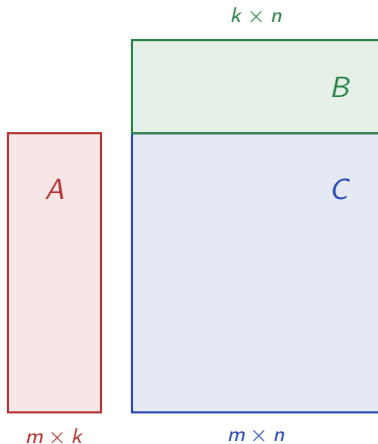
256×256 fp64:  $\approx 0.5$  MB

Typical L1 cache: 16–64 KB

$\Rightarrow 8 \times - 32 \times$  **larger than L1**

**Fix:** *tile* the loop — bring in one smaller  $C$  tile and keep it resident in L1.

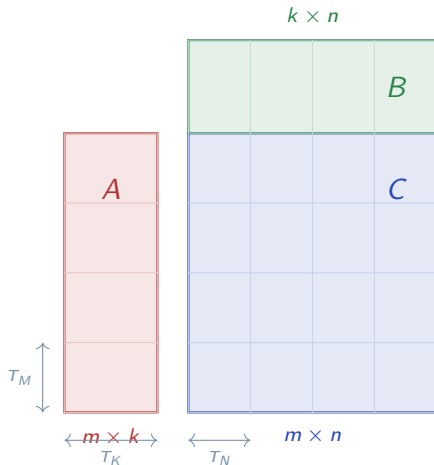
# C-Stationary Tiling: Step by Step



$$C = A \cdot B$$

For large  $m, n, k$  — none fit in L1.

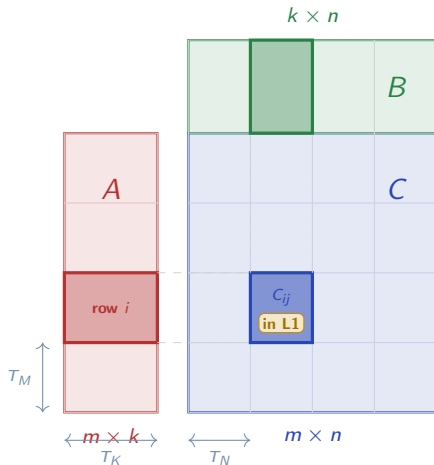
# C-Stationary Tiling: Step by Step



**Tile shape:**  $T_M \times T_N \times T_K$

Each matrix divided into a grid of tiles.

# C-Stationary Tiling: Step by Step

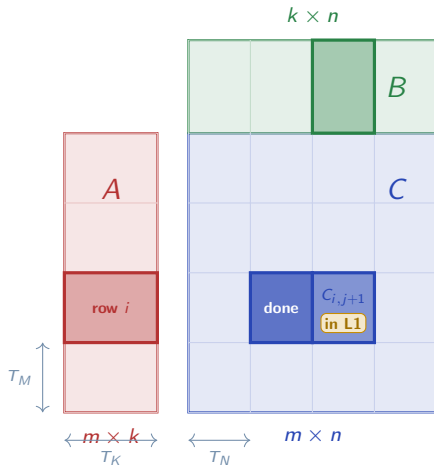


Fix  $C_{ij}$  in L1.

Stream  $A$  row-band and  $B$  col-band through it.

$$C_{ij} += A_{\text{row } i} \times B_{\text{col } j}$$

# C-Stationary Tiling: Step by Step



$C_{ij}$  done — move to  $B$  col  $j+1$ .

**A row-band stays in L1.**

# Asymmetric Precision $\Rightarrow$ Asymmetric Tiles

$$\#A \text{ Bytes} = T_M \cdot k \cdot A_p$$

$$\#B \text{ Bytes} = T_N \cdot k \cdot B_p$$

$$\Rightarrow T = \frac{MN}{T_M T_N} (T_M \cdot k \cdot A_p + T_N \cdot k \cdot B_p)$$

$$T = MNK \left( \frac{A_p}{T_N} + \frac{B_p}{T_M} \right)$$

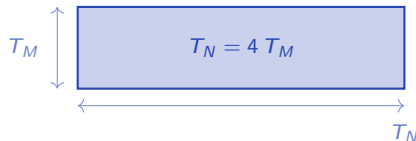
Find best ratio:

$$\Rightarrow \frac{T_N^*}{T_M^*} = \frac{A_p}{B_p} = \frac{1}{\rho}$$

$$\rho = 1 \text{ (fp32} \times \text{fp32)}$$



$$\rho = \frac{1}{4} \text{ (fp32} \times \text{fp8)}$$

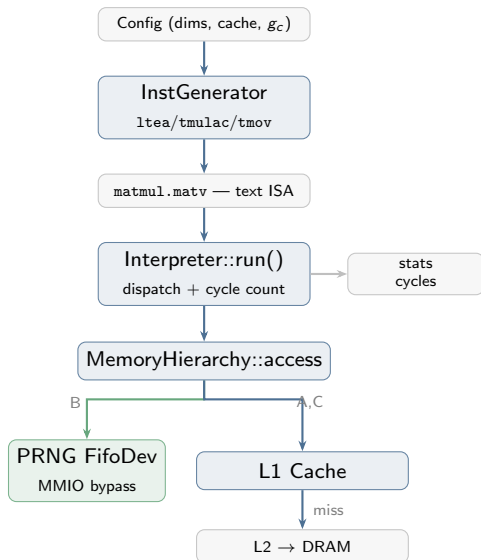


Same cache budget  $S$  — wider tile, better B reuse.

# The Simulator We Built

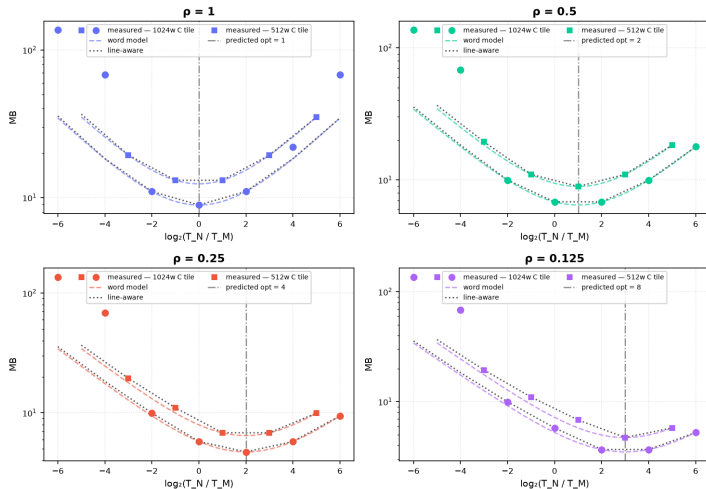
## Three main components:

- **InstGenerator** — compiles tile ops to a text ISA (ltea/tmulac/tmov)
- **Interpreter** — dispatches instructions, counts cycles per access
- **MemoryHierarchy** — configurable L1/L2/DRAM with per-access tracing



# Traffic Minimum at the Predicted Tile

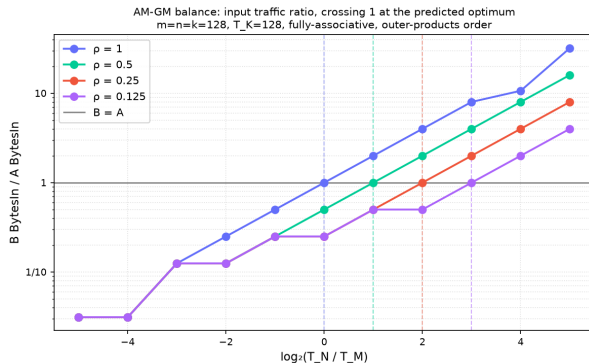
L1 reads (BytesIn) vs paper prediction — fully-associative cache  
 $m=n=k=256$ ,  $T_K=256$ , outer-products order



Each curve shows L1 reads as a function of tile aspect ratio  $T_N/T_M$ .

**Dashed line:** paper's prediction  
 $T_N/T_M = 1/\rho$ .

# B/A Balance: Exact Confirmation



Each curve is the ratio of B reads to A reads as  $T_N/T_M$  varies.

## Traffic saved vs. square tile:

| $\rho$ | saving         |
|--------|----------------|
| 1      | 0% (symmetric) |
| 0.5    | 0%             |
| 0.25   | 18%            |
| 0.125  | 36%            |

The paper minimizes *bytes transferred*:

$$T_{\text{traffic}} = MNK \left( \frac{A_P}{T_N} + \frac{B_P}{T_M} \right)$$

**But cycles  $\neq$  bytes.**

We need a *cycle-accurate* model.

# Enter the PRNG FIFO

## Enter the PRNG FIFO

B elements are generated *on-chip* by a hardware random number generator, streamed directly into the compute unit.

**B never touches the cache.**

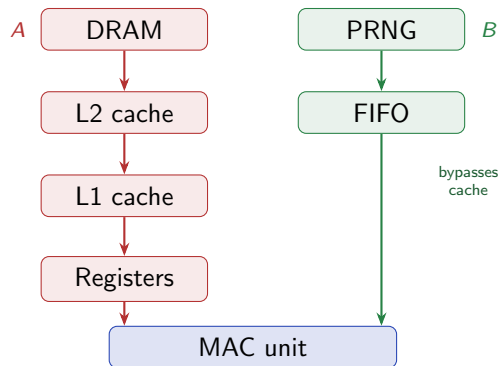
**B has no memory address.**

The  $B_P/T_M$  traffic term **vanishes**.

Instead, B introduces a new cost: on-chip *generation time*.

$$\text{generation cost} = g_c \text{ cycles / element}$$

# The PRNG FIFO: B Without Memory



**Random matmul:**  $C = A \cdot B$  where  $B$  is a pseudo-random matrix generated on-chip.

**Use cases:** random projections, privacy-preserving ML, approximate algorithms.

## Two independent bottlenecks

A-bound: load A from memory

B-bound: wait for PRNG ( $g_c$  cy/elem)

## The PRNG FIFO: B Without Memory

Runtime = ?

## The PRNG FIFO: B Without Memory

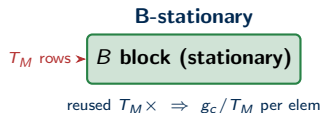
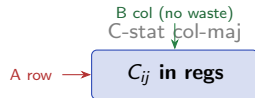
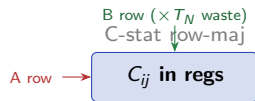
Runtime = whichever is slower

# Which Loop Order Fits the FIFO?

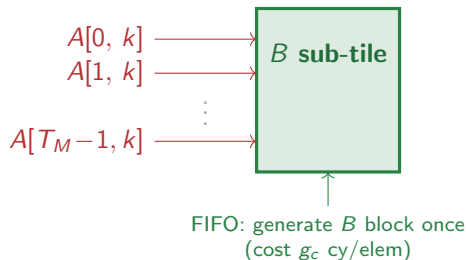
**Problem:** B elements emerge from the PRNG in a **fixed order** — you cannot skip or replay elements.

Three loop structures are possible:

| Mode           | In regs       | B cost / elem            |
|----------------|---------------|--------------------------|
| C-stat row-maj | $C_{ij}$ tile | $g_c \times T_N$ (waste) |
| C-stat col-maj | $C_{ij}$ tile | $g_c$                    |
| B-stationary   | $B_{ij}$ tile | $g_c / T_M$              |



## B-Stationary: $T_M$ -Fold Register Reuse



**Loop order:** B outer, A inner. Each B block is fetched *once*.  $\rightarrow$  Each B element is used  $T_M$  times — generation cost amortized:

|           | $g_c=0$           | $g_c=10$           | $g_c=100$         |
|-----------|-------------------|--------------------|-------------------|
| B-stat    | ref               | ref                | <b>ref</b>        |
| C col-maj | $1.9 \times$ fstr | $1.25 \times$ fstr | $7.7 \times$ slwr |
| C row-maj | $1.6 \times$ slwr | $6.3 \times$ slwr  | $61 \times$ slwr  |

# Naive Cycle Model: Two Cost Terms

Same reuse argument as traffic, but in **cycles**:

Naively assuming the two costs **add**:

$$\frac{T}{MNK} = \frac{C_A}{T_N} + \frac{g_c}{T_M}$$

# FIFO is Async: Two Costs in Parallel

The FIFO generates B **in the background** while the core loads A from L1.

The two costs **overlap** — runtime is determined by whichever finishes last:

$$\frac{T}{MNK} = \max \left\{ \frac{C_A}{T_N}, \frac{g_c}{T_M} \right\}$$

**Two regimes:**

$g_c/T_M < C_A/T_N$ : A-load bound

$g_c/T_M > C_A/T_N$ : B-gen bound

## Replacing $C_A/T_N$ with Measured $\alpha(T_M, T_N)$

The  $C_A/T_N$  term assumes:

$C_A$  = fixed cost per A element from L1  
 $T_N$ -fold reuse divides it out cleanly

**Problem:** actual cycles/MAC depends on tile shape through cache residency, line utilization, and access patterns — not a simple constant divided by  $T_N$ .

**Solution:**

$$\frac{T}{MNK} = \max \left\{ \alpha(T_M, T_N), \frac{g_c}{T_M} \right\}$$

## Isolating $\alpha$ : Set $g_c = 0$

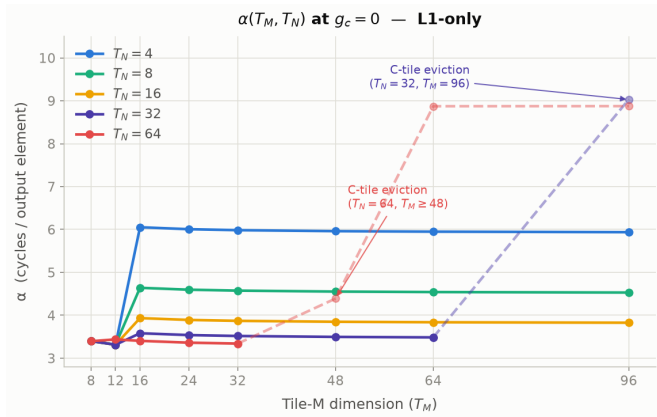
**Key trick:** set  $g_c = 0$

The model collapses:

$$\frac{T}{MNK} = \max \left\{ \alpha(T_M, T_N), \frac{0}{T_M} \right\} = \alpha(T_M, T_N)$$

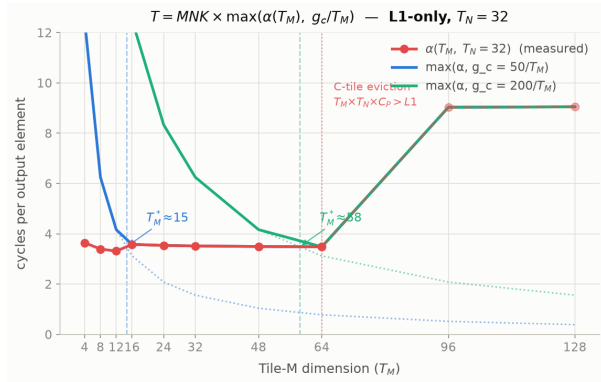
# Calibration: Building the $\alpha$ Table

- Small  $T_N$ :  $\alpha$  spikes when the A tile overflows L1
- Large  $T_N$ :  $\alpha$  stays low until the C tile overflows L1
- $\alpha$  decreases with  $T_N$  (more column reuse)



# Using the $\alpha$ Table: Predicting the Optimal Tile

The optimal tile is where the two costs are balanced:  $\alpha(T_M^*, T_N^*) \approx g_c / T_M^*$ .



# Roofline Validation: Setup

## Question:

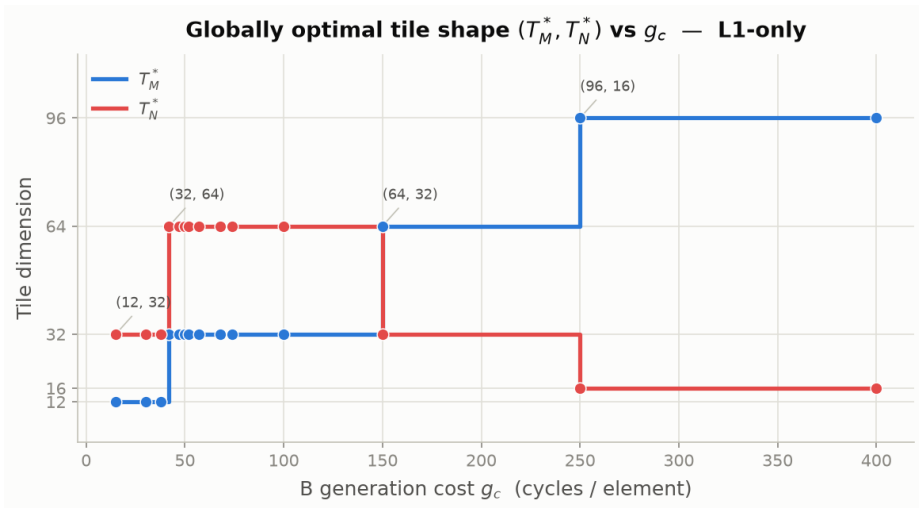
calibrate  $\alpha$  once at  $g_c = 0$ ; does the model predict the best tile correctly at any new  $g_c$ ?

# Roofline Validation: Results

| All 108 conditions     |                  |        |
|------------------------|------------------|--------|
| $(T_M^*, T_N^*)$ exact | <b>108 / 108</b> | (100%) |

**100% exact match across all hardware configurations and  $g_c$  values.**

# Impact of Tile Selection: Optimal vs. Square



# Impact of Tile Selection: Optimal vs. Square

## Question:

Does choosing the right ( $T_M, T_N$ ) actually matter?

## Comparing:

- Optimal asymmetric tile ( $T_M^*, T_N^*$ )
- Square tile ( $T_M = T_N$ , fixed)

## Speedup from correct tile:

- $g_c = 100$ : **20–65%** faster
- $g_c = 250$ : **~85%** faster
- $g_c = 400$ : **up to 90%** faster

# Impact of Tile Selection: Optimal vs. Square

L1-only — empirically measured performance (E8 sweep)

